JavaScript: Variable

Basic Understanding

- Variable is a named memory address that used for storing data
- Remember that JavaScript language is **case-sensitive**
- Keywords for declaring a variable: `let`, `const`, `var`
- Assigning a value by using = symbol
- https://www.programiz.com/

Use variable for storing data

Storing a number value

```
let x = 4;
let y = 3;
let z = x + y;

// Use console.log() to show value inside the variable
console.log(z);
```

Stroring a string value

```
let a = "Ozzy";
let b = "Orange";
let c = "Gripen"
```

Storing a boolean value

```
let i = true;
let j = false;
let k = i && j;
```

Variable name should be meaningful

```
let isOpen = true;
let firstName = "Ozzy";
let model = "Gripen";
let numberOfBoms = 13;
```

Storing an object

```
let bassist = { firstName: "Anddy", lastName: "Summer", age: 68, isAlive: true };
let singer = bassist;
```

Storing an array

```
let numbers = [2, 5, 9, 17];
let planeModels = ["F-16", "Gripen"];
let manyTypes = [
    "Ozzy",
    23,
    {
        isGood: "good"
    }
];
```

Variable name

- underscore => Yes
- dollar sign => Yes
- reserved words => No
- cannot started with a number

```
let number = 12;
let _cars = ["BMW", "Volvo"];
let $models = "Gripen";
let guitarist1 = "John Mayer";
```

We cannot use reserved words for a variable name

- `function`
- `false`
- `let`

etc.

The differents between `let`, `const`, and `var`

- **Scope**
  - Where these variales are available for use:
    - *function scoped* or *local scoped* or *block scoped*
- **Redeclaration**: recreating a variable
- **Reassignment**: assigning a new value to an already created variable
- **Initialization**: to assigne an initial value to a variable

Play with `var`

```
var global = "global"

function test() {
    var f1 = "function";
    console.log(f1);
}

if(true) {
    var block = "block";
}

go();

console.log(global)
console.log(block);
// console.log(f1);
```

> *`var` is gobally and function scoped*

Play with `let`

```
let global = "global"

function test() {
    let f1 = "function";
    console.log(f1);
}

if(true) {
    let block = "block";
}

go();

console.log(global)
// console.log(block);
// console.log(f1);
```

Play with `const`

```
const global = "global"

function test() {
    const f1 = "function";
    console.log(f1);
}

if(true) {
    const block = "block";
}

go();

console.log(global)
```

```
// console.log(block);
// console.log(f1);
```

> *`let` and `const` is block scoped*

`var` can be redeclared, `let` and `const` cannot be dereclared

```
var ok = "ok";
var ok = "great";
console.log(ok);
```

Reassignment is possible with `var` and `let`

```
var name = "John";
name = "Ben";

console.log(name);
```

> *reassignment is possible for `var` and `let`*

> *reassignment isn't possible for `const`*

Initialization

```
let name1;
console.log(name1);

// const name2;
// console.log(name2);
```

>

> *`var` and `let` are initializaed to `undefined`*

> *`const` required initialized value*

Summary of the differences

- **Scope**:
  - `var` is globally and function scoped
  - `let` and `const` is block scoped
- **Redeclaration**:
  - `var` can be redeclared
  - `let` and `const` cannot be redeclared
- **Reassignment**:
  - reassignement is possible for `var` and `let`
  - reassignement is not possible for `const`
- **Initialization**:
  - `var` and `let` are initialized to `undefined`
  - `const` must be initialized

Key takeaways:

- Use `const` by default
- Use `let` when values need to change
- Avoid `var` in modern JavaScript
- Understand **primitive** vs **reference** types